

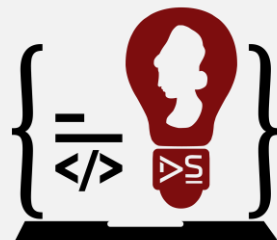


CICLO 1/ 2024 - ASIGNATURA:

LÓGICA DE PROGRAMACIÓN

UNIVERSIDAD DE EL SALVADOR - FMOcc

Departamento de Ingeniería y Arquitectura



Ingeniería en Desarrollo
de Software



```
static bool init(CURL *conn, char *url, struct curl_slist *headers)
{
    CURLcode code;

    conn = curl_easy_init();

    if (conn == NULL)
    {
        fprintf(stderr, "Failed to create CURL connection\n");
        exit(EXIT_FAILURE);
    }

    code = curl_easy_setopt(conn, CURLOPT_ERRORBUFFER,
        errorBuffer);
    if (code != CURLE_OK)
    {
        fprintf(stderr, "Failed to set error buffer [%d]\n",
            code);
        return false;
    }

    code = curl_easy_setopt(conn, CURLOPT_URL, url);
    if (code != CURLE_OK)
    {
        fprintf(stderr, "Failed to set URL [%s]\n", url);
        return false;
    }

    code = curl_easy_setopt(conn, CURLOPT_FOLLOWLOCATION,
        1L);
    if (code != CURLE_OK)
    {
        fprintf(stderr, "Failed to set redirect option [%s]\n", url);
        return false;
    }

    code = curl_easy_setopt(conn, CURLOPT_WRITEFUNCTION,
        writer);
    if (code != CURLE_OK)
    {
        fprintf(stderr, "Failed to set writer [%s]\n", url);
        return false;
    }

    code = curl_easy_setopt(conn, CURLOPT_WRITEDATA,
        &buffer);
    if (code != CURLE_OK)
    {
        fprintf(stderr, "Failed to set write data [%s]\n", url);
        return false;
    }

    return true;
}
```

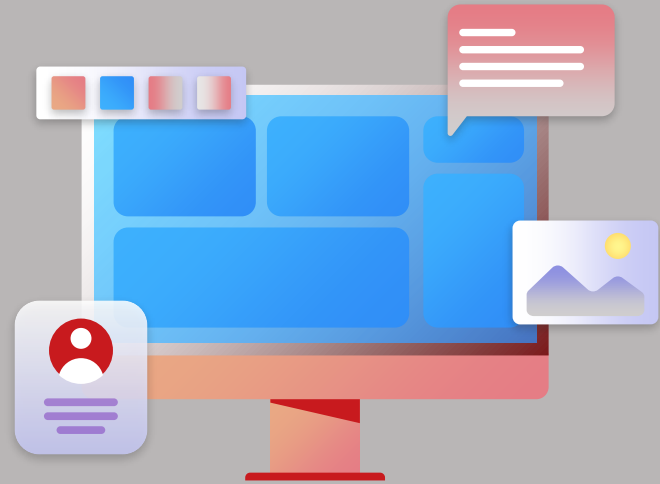
Avisos y Anuncios semana 9

- Actividades a Realizar
 - Desarrollar el Examen Corto
 - Participación en el Foro
 - Leer los contenidos de esta semana



Funciones, Arreglos, Estructuras, Memoria, Archivos, Listas

Unidad 1





Agenda

/1.5.5 Archivos Binarios

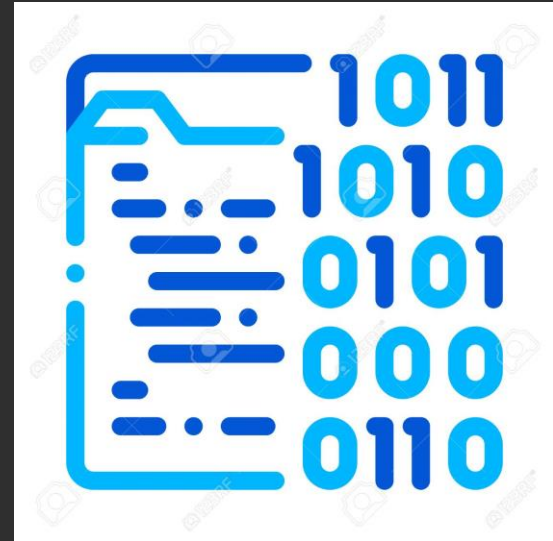
**/1.5.6 Funciones para
acceso Aleatorio.**

**/1.5.7 Datos Externos al
Programa con
Argumentos.**

**/1.6.1 Fundamentos
Teóricos de Listas
Simples.**



1.5.5 Archivos Binarios.



Archivos Binarios.



- Los archivos binarios almacenan flujos de bits, sin prestar atención a los códigos ASCII o a la separación de espacios. Son adecuados para almacenar objetos. Sin embargo, el uso de los archivos binarios requiere usar la dirección de una posición de almacenamiento.

Archivos Binarios.



- En general, usaremos ficheros de texto para almacenar información que pueda o deba ser manipulada con un editor de texto. Un ejemplo es un fichero fuente C++. Los ficheros binarios son más útiles para guardar información cuyos valores no estén limitados. Por ejemplo, para almacenar imágenes, o bases de datos. Un fichero binario permite almacenar estructuras completas, en las que se mezclen datos de cadenas con datos numéricos.

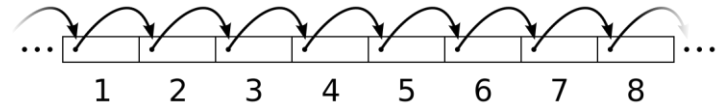
Ejemplo Archivos Binarios.

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;
struct tipoRegistro {
    char nombre[32];
    int edad;
    float altura;
};
```

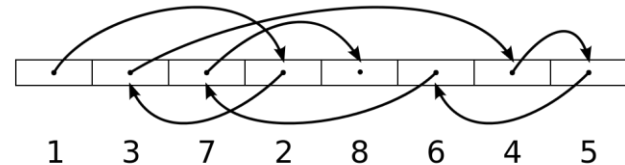
```
int main() {
    tipoRegistro pepe;
    tipoRegistro pepe2;
    ofstream fsalida("prueba.dat",
        ios::out | ios::binary);
    strcpy(pepe.nombre, "Jose Luis");
    pepe.edad = 32;
    pepe.altura = 1.78;
    fsalida.write(reinterpret_cast<char *>(&pepe),
        sizeof(tipoRegistro));
    fsalida.close();
    ifstream fentrada("prueba.dat",
        ios::in | ios::binary);
    fentrada.read(reinterpret_cast<char *>(&pepe2),
        sizeof(tipoRegistro));
    cout << pepe2.nombre << endl;
    cout << pepe2.edad << endl;
    cout << pepe2.altura << endl;
    fentrada.close();
    return 0;
}
```


1.5.6 Funciones para Acceso Aleatorio.

Acceso secuencial



Acceso aleatorio



Funciones para Acceso Aleatorio.



- El término acceso a archivos se refiere al proceso de recuperar datos de un archivo. Existen dos tipos de acceso a los archivos: acceso secuencial y acceso aleatorio
- Aunque los caracteres en el archivo estén almacenados en forma secuencial, esto no nos obliga a tener acceso secuencial a ellos. De hecho, podemos saltar caracteres y leer un archivo organizado en forma secuencial de una manera no secuencial

Funciones para Acceso Aleatorio.



- En el acceso aleatorio, cualquier carácter en el archivo abierto puede leerse en forma directa sin tener que leer primero en forma secuencial todos los caracteres almacenados antes que él.
- Para proporcionar acceso aleatorio a los archivos, cada objeto **ifstream** crea en forma automática un marcador de posición de archivo. Este marcador es un número entero largo que representa un desplazamiento desde el principio de cada archivo y le sigue la pista al lugar desde donde se va a leer o a escribir el siguiente carácter.



Funciones de marcadores de posición del archivo

- Las funciones **seek()** permiten al programador moverse a cualquier posición en el archivo. Las funciones **seek()** requieren dos argumentos: el primero es el desplazamiento, como un número entero largo, en el archivo; el segundo es desde dónde se va a calcular el desplazamiento, lo que es determinado por el modo. Las tres alternativas posibles para el modo son **ios::beg**, **ios::cur** e **ios::end**, los cuales denotan el principio, la posición actual y el final del archivo, respectivamente.

Nombre	Descripción
<code>seekg(offset, mode)</code>	Para archivos de entrada, se mueve a la posición de desplazamiento indicada por el modo.
<code>seekp(offset, mode)</code>	Para archivos de salida, se mueve a la posición de desplazamiento indicada por el modo.
<code>tellg(void)</code>	Para archivos de entrada, devuelve el valor actual del marcador de posición del archivo.
<code>tellp(void)</code>	Para archivos de salida, devuelve el valor actual del marcador de posición del archivo.

Funciones de marcadores de posición del archivo

- A diferencia de las funciones **seek()** que mueven el marcador de posición del archivo, las funciones **tell()** devuelven el valor de desplazamiento del marcador de posición del archivo

Nombre	Descripción
<code>seekg(offset, mode)</code>	Para archivos de entrada, se mueve a la posición de desplazamiento indicada por el modo.
<code>seekp(offset, mode)</code>	Para archivos de salida, se mueve a la posición de desplazamiento indicada por el modo.
<code>tellg(void)</code>	Para archivos de entrada, devuelve el valor actual del marcador de posición del archivo.
<code>tellp(void)</code>	Para archivos de salida, devuelve el valor actual del marcador de posición del archivo.

1.5.7 Datos Externos al Programa con Argumentos.



Datos Externos al Programa con Argumentos. ●

- Un objeto de flujo de archivo puede utilizarse como un argumento de función. El único requisito es que el parámetro formal de la función sea una referencia al flujo apropiado, ya sea **ifstream&** u **ofstream&**.

Datos Externos al Programa con Argumentos. ●

```
#include <iostream>
#include <fstream>
#include <cstdlib>
#include <string>
using namespace std;
int main()
{
    string nombre_archivo = "lista.dat"; // aquí está el archivo con el que
    estamos trabajando
    void inOut(ofstream&); // prototipo de la función
    ofstream archivo_sal;
    archivo_sal.open(nombre_archivo.c_str());
    if (archivo_sal.fail()) // comprueba una apertura exitosa
    {
        cout << "\nEl archivo de salida " << nombre_archivo << " no se abrió
        con éxito"
            << endl;
        exit(1);
    }
    inOut(archivo_sal); // llama a la función
    return 0;
}
```


Datos Externos al Programa con Argumentos. ●

```
void inOut(ofstream& salArchivo)
{
    const int NUMLINEAS = 5; // número de líneas de texto
    string linea;
    int cuenta;

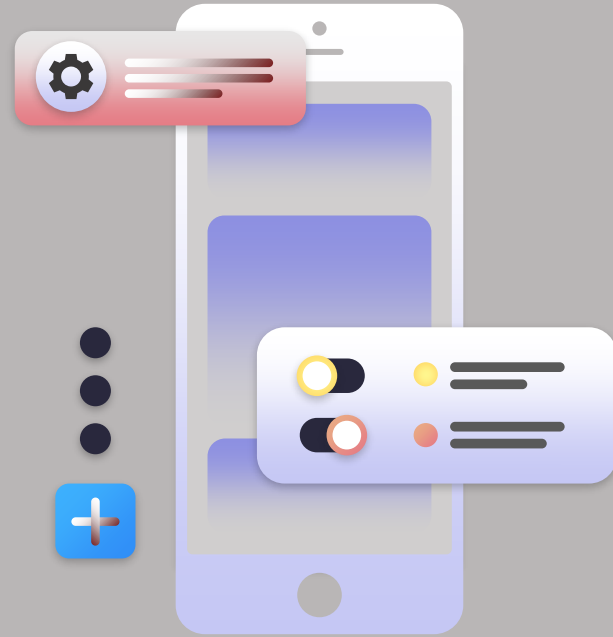
    cout << "Por favor introduzca cinco líneas de texto:" << endl;
    for (cuenta = 0; cuenta < NUMLINEAS; cuenta++)
    {
        getline(cin, linea);
        salArchivo << linea << endl;
    }
    cout << "\nEl archivo se ha escrito con éxito." << endl;
    return;
}
```

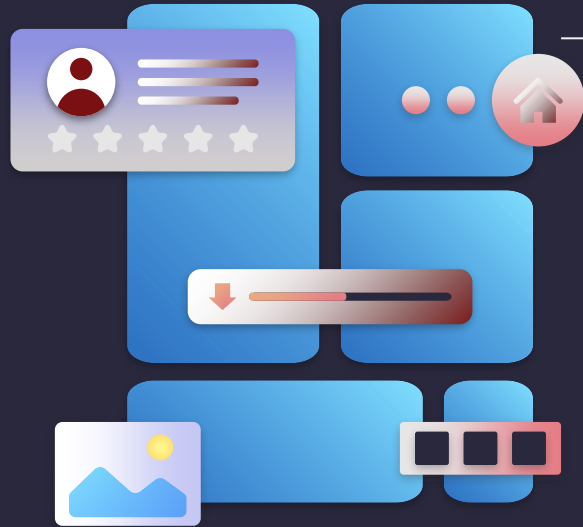
Datos Externos al Programa con Argumentos. ●

- En el programa anterior dentro de `main()`, el archivo es un objeto ostream llamado `archivo_sal`. Este objeto es transmitido a la función `inOut()` y es aceptado como el parámetro formal llamado `salArchivo`, el cual es declarado como una referencia a un tipo de objeto ostream. La función `inOut()` usa entonces su parámetro de referencia `archivo_sal` como un nombre de flujo de archivo de salida de la misma manera que `main()` usaría el objeto de flujo `salArchivo`.

1.6

Introducción al Uso de Listas Simple.





/INTRODUCCIÓN

Al contrario que las estructuras de datos estáticas en las que su tamaño en memoria se establece durante la compilación y permanece inalterable durante la ejecución del programa, las estructuras de datos dinámicas crecen y se contraen a medida que se ejecuta el programa.

1.6.1 Fundamentos Teóricos.



Fundamentos Teóricos.

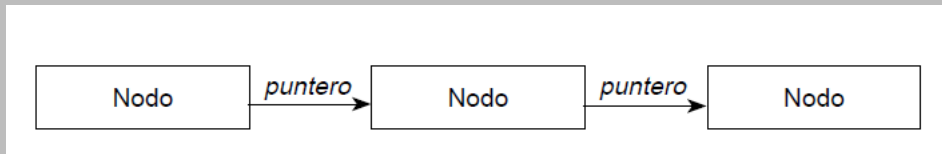


- Una lista enlazada es una colección o secuencia de elementos dispuestos uno detrás de otro, en la que cada elemento se conecta al siguiente por un «enlace» o «puntero». La idea básica consiste en construir una lista cuyos elementos llamados nodos se componen de dos partes o campos: la primera parte o campo contiene la información y es, por consiguiente, un valor de un tipo genérico (denominado Dato, TipoElemento, etc.) y la segunda parte o campo es un puntero que apunta al siguiente elemento de la lista.

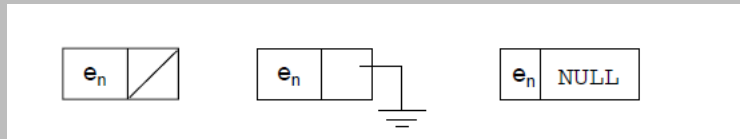


Fundamentos Teóricos.

- Los enlaces se representan por flechas para facilitar la comprensión de la conexión entre dos nodos. Los enlaces también sitúan los nodos en una secuencia. El primer nodo se enlaza al segundo, el segundo se enlaza al tercero, y así sucesivamente, hasta llegar al último nodo.



- El último ha de ser representado de forma diferente para significar que este nodo no se enlaza a ningún otro



/Recordatorios

/Realizar la actividad evaluada de la semana



/Participacion en foros



/Leer los contenidos vistos en clase



/Codificar los ejemplos expuestos



/GRACIAS!

/ALGUNA PREGUNTA?

